

M14 / PROTECTED ANSWERS

M14 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 4

1. DE14-01 - validation 4

2. DE14-02 - validation 4

3. DE14-03 - validation 4

4. DE14-04 - validation 4

5. DE14-05 - validation 4

6. DE14-06 - validation 4

7. DE14-07 - validation 4

8. DE14-08 - validation 5

9. DE14-09 - validation 5

10. DE14-10 - validation 5

11. DE14-11 - validation 5

12. DE14-12 - validation 5

M14 | WORKED VALIDATION APPENDIX 5

A01 - DE14-01 | Batch vs streaming 5

A02 - DE14-02 | Events, brokers and topics 6

A03 - DE14-03 | Producers and keys 6

A04 - DE14-04 | Consumers and safe processing 6

A05 - DE14-05 | Partitions and ordering 7

A06 - DE14-06 | Offsets and replay 7

A07 - DE14-07 | Consumer groups and scaling 8

A08 - DE14-08 | Delivery semantics/idempotency 8

A09 - DE14-09 | Serialization and schema evolution 8

A10 - DE14-10 | Python producer/consumer workflow 9

A11 - DE14-11 | Spark Structured Streaming + Kafka 9

A12 - DE14-12 | Lag/failures/operations 10

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after saving the closed-book attempt. Repair one weak lesson, close this guide, then take a changed retry.

1. DE14-01 - validation

A batch/stream decision should start from business latency and recovery needs, not tool prestige. For example, nightly payroll has a natural business boundary and usually benefits from batch simplicity. Parcel fraud alerts have a low-latency requirement, so events should be available continuously. A hybrid system can stream raw events into durable storage and still compute some official reports in batch.

2. DE14-02 - validation

A topic is a retained append-only log. Brokers store topic partitions. Consumers read without deleting for other groups. Therefore a second group can replay retained events. If a new consumer sees only fresh events, first check group/starting position before claiming the broker deleted history.

3. DE14-03 - validation

Use the business entity whose order matters as the message key. For order lifecycle events that is usually `order_id`. Delivery callbacks prove broker outcome; an immediate local print does not. Stable keys help keep related events in one partition while partition count is stable.

4. DE14-04 - validation

The safer at-least-once pattern is durable effect first, commit after. That leaves a replay window, so the sink must be idempotent. Commit-first leaves a loss window. External APIs are not automatically covered by Kafka transactions.

5. DE14-05 - validation

Kafka ordering is partition-local. Four partitions are four ordered logs, not one globally ordered stream. A key that sends one entity to one partition gives per-entity order, but concentrated keys can produce skew.

6. DE14-06 - validation

Offsets are partition-local positions. A committed offset is a group bookmark. Replaying means choosing an earlier retained position; it does not modify the log. `auto.offset.reset` applies when there is no valid committed position, not as a universal rewind switch.

7. DE14-07 - validation

Consumers sharing `group.id` share partition work. Different group IDs independently read the topic. With 6 partitions, more than 6 consumers cannot all own one of those partitions at the same moment in the normal group model.

8. DE14-08 - validation

Producer idempotence prevents duplicates caused by retrying the same producer sequence under supported settings. It does not deduplicate two intentional business events. Kafka transactions can atomically coordinate Kafka records/offsets, but an external database/API needs its own idempotency/coordination design.

9. DE14-09 - validation

A schema is a contract over serialized bytes. Adding an optional field with a compatible default is generally safer than renaming a required field or changing a number into a formatted string. Exact compatibility depends on the schema format and registry policy.

10. DE14-10 - validation

The Python Producer is asynchronous: use delivery callbacks and flush/poll lifecycle correctly. The Consumer needs stable group identity, explicit progress semantics, validation/quarantine and close(). A healthy process is not proof of reconciled data.

11. DE14-11 - validation

Spark Structured Streaming reads Kafka as a source but tracks query progress/checkpoint state using Spark mechanisms. A checkpoint is part of recovery correctness. Deleting it can cause reprocessing or changed state semantics, especially with a non-idempotent sink.

12. DE14-12 - validation

Consumer lag is per-partition distance between produced/log-end progress and consumer progress. Growing lag means the application is falling behind, but the cause could be skew, slow sink, poison records, insufficient capacity or rebalance churn.

M14 | WORKED VALIDATION APPENDIX

Use this appendix only after a saved attempt. It turns short validation points into concrete solved examples, failure checks and changed retries.

A01 - DE14-01 | Batch vs streaming

Concrete solved example

Use streaming for low-latency route alerts and batch for the official nightly billing total. The stream can land durable events that the batch later reconciles; one mode does not need to replace the other.

Worked proof / evidence

Alert SLO: seconds/minutes; billing boundary: one business day + reconciliation close.

Common mistake

Choosing streaming for billing just because Kafka exists.

Validation

State latency, completeness, replay and correction requirements for each output.

Changed retry

Change billing to hourly provisional + nightly final and redraw the boundary.

A02 - DE14-02 | Events, brokers and topics

Concrete solved example

Two independent applications use different consumer group IDs. The topic retains records; one group reading does not delete them for the other group.

Worked proof / evidence

group.analytics reads offsets independently from group.fraud

Common mistake

Using same group ID and expecting both apps to independently receive every record.

Validation

Show two group positions on the same topic and a replay from an earlier retained offset.

Changed retry

Add a third application that must see every event; choose its group ID.

A03 - DE14-03 | Producers and keys

Concrete solved example

Use order_id as the key for order lifecycle events so one order normally maps to one partition and keeps per-order order while partition count is stable. Watch for hot orders/skew and partition-count changes.

Worked proof / evidence

key=b"O1001" -> partition p; callback records broker delivery outcome

Common mistake

Using a random key when entity order matters; treating local send() return as broker acknowledgement.

Validation

Produce multiple events for one order and inspect partition/offset order and delivery callbacks.

Changed retry

Use customer_id instead and explain what ordering changes.

A04 - DE14-04 | Consumers and safe processing

Concrete solved example

For an external sink: poll -> decode/validate -> durable idempotent sink write -> validate effect -> commit offset. If the sink write fails, do not quarantine a valid record and do not commit it.

Worked proof / evidence

valid event + PermissionError on sink => exception/retry/stop; committed offset remains before failed event

Common mistake

Broad except Exception that quarantines infrastructure failures and commits progress.

Validation

Inject invalid JSON: quarantine durably then commit under poison policy. Inject sink failure on valid data: no commit.

Changed retry

Inject commit failure after successful sink write and prove replay is safe because sink is idempotent.

A05 - DE14-05 | Partitions and ordering

Concrete solved example

Kafka preserves order within each partition. A merged dashboard can display B before A if A and B belong to different partitions or the downstream merge has its own timing/order.

Worked proof / evidence

partition0: A1,A2; partition1: B1,B2; merged display may interleave

Common mistake

Assuming a topic is globally ordered.

Validation

Record topic/partition/offset with events and reason about order only within a partition unless stronger design exists.

Changed retry

Re-key two related events so they land together and compare observed order.

A06 - DE14-06 | Offsets and replay

Concrete solved example

auto.offset.reset=earliest applies when the group has no valid committed position. An existing healthy group continues from its commits; explicit reset/seek/new group is needed for replay.

Worked proof / evidence

group has committed p0=120 -> earliest setting does not rewind it to 0

Common mistake

Changing config and expecting an established group to rewind automatically.

Validation

Inspect committed offsets before/after restart; perform controlled replay with a fresh group or explicit reset.

Changed retry

Replay only one partition from an earlier offset and document duplicate-sink protection.

A07 - DE14-07 | Consumer groups and scaling

Concrete solved example

With 6 partitions and 10 consumers in one normal group, at most 6 consumers can own a partition at once; others are idle. More consumers do not create more partitions.

Worked proof / evidence

partitions=6; active owners<=6

Common mistake

Scaling consumers without checking partition count or skew.

Validation

Inspect assignment and lag per partition before/after scaling.

Changed retry

Make one partition hot; explain why 6 consumers may still not solve skew.

A08 - DE14-08 | Delivery semantics/idempotency

Concrete solved example

Producer idempotence protects against retry duplicates from the same producer session under supported settings; it does not deduplicate two intentional business events. For external DB/API sinks, use business idempotency keys/constraints and commit-after-effect.

Worked proof / evidence

DB table unique(event_id); replayed Kafka record -> insert conflict/no-op -> commit offset

Common mistake

Saying Kafka transactions automatically make an external database exactly-once.

Validation

Crash after DB commit before Kafka commit; replay must not duplicate business effect.

Changed retry

Use an external HTTP API with idempotency-key support and define retry behavior.

A09 - DE14-09 | Serialization and schema evolution

Concrete solved example

Add a new optional field with compatible/default behavior when policy allows; renaming/removing required fields or changing number to formatted string is breaking unless compatibility policy explicitly handles it.

Worked proof / evidence

old consumer ignores/uses default for optional field; required rename fails compatibility check

Common mistake

Equating valid JSON with schema compatibility.

Validation

Run/inspect registry compatibility rule or simulate old/new reader expectations.

Changed retry

Change enum/domain values and decide whether schema format/policy considers it compatible.

A10 - DE14-10 | Python producer/consumer workflow

Concrete solved example

Producer uses callbacks plus poll/flush lifecycle. Consumer has stable group ID, explicit validation/quarantine, idempotent sink, explicit commit policy and close().

Worked proof / evidence

produce -> callback ack; consume -> validate -> sink -> commit; invalid JSON -> quarantine -> commit

Common mistake

Printing “sent” before delivery callback; enable.auto.commit with unsafe external sink.

Validation

Reconcile produced IDs to sink IDs + quarantine IDs and inspect committed positions.

Changed retry

Kill consumer after sink write but before commit and prove replay safety.

A11 - DE14-11 | Spark Structured Streaming + Kafka

Concrete solved example

Parse raw Kafka bytes with explicit schema; classify malformed JSON, missing key, nonnumeric/negative amount into quarantine while retaining topic/partition/offset/raw payload; valid + quarantine must reconcile. Use a stable checkpoint for restart.

Worked proof / evidence

```
valid_count + quarantine_count == input_count  
quarantine retains topic,partition,offset,value
```

Common mistake

Connectivity-only script described as full validation; deleting checkpoint casually.

Validation

Feed one valid, one malformed, one negative amount and restart from same checkpoint; compare outcomes.

Changed retry

Change checkpoint location intentionally and explain why that is a new query state, not normal resume.

A12 - DE14-12 | Lag/failures/operations

Concrete solved example

Lag is per-partition log-end minus consumer position. Diagnose skew, slow sink, poison record loop, rebalance churn or insufficient capacity before scaling blindly.

Worked proof / evidence

p0 lag=0, p1 lag=5000 with same group -> inspect p1 throughput/key skew/sink latency

Common mistake

Looking only at total lag or adding consumers beyond partition count.

Validation

Track lag slope per partition, processing rate, errors/rebalances and sink latency during incident.

Changed retry

Simulate one poison record that retries forever and show how lag pattern differs from uniform overload.